

Case 6: Parametrize — Test Multiple Posts

Intermediate

Run the same test logic against multiple post IDs without duplicating code.

New concept: `@pytest.mark.parametrize`

API Endpoint

GET `https://jsonplaceholder.typicode.com/posts/{id}`

What You Will Learn

- `@pytest.mark.parametrize` to run one test with multiple inputs
- How pytest generates a separate test case per parameter
- Validating that each post's id matches the requested id
- Keeping tests DRY (Don't Repeat Yourself)

Study Plan

Step 1 — The problem parametrize solves

Without parametrize, testing posts 1, 2, and 3 means writing 3 identical tests. Parametrize lets you write one test and run it N times with different inputs.

Step 2 — Basic parametrize syntax

Place `@pytest.mark.parametrize('param_name', [val1, val2, val3])` above your test function. The `param_name` becomes an argument your test receives.

Step 3 — Build the URL dynamically

Use an f-string to inject the `post_id` into the URL: `f'https://jsonplaceholder.typicode.com/posts/{post_id}'`. Each run uses a different id.

Step 4 — Assert id matches

The API returns the post with that id. Assert that `response.json()['id']` equals the `post_id` you passed in — a perfect cross-check.

Step 5 — Read the output

When you run `pytest -v`, you will see one line per parameter value: `test_post_by_id[1]`, `test_post_by_id[2]`, `test_post_by_id[3]`. Each is a separate test.

Your Task

Write a parametrized pytest test that calls GET `/posts/{id}` for post IDs: 1, 2, 3, 5, 10

Each test run must verify:

1. Status code is 200
2. The returned post id matches the requested id
3. The post contains a non-empty title (hint: `len()` or truthy check)

BONUS: Add a `pytest.param` entry that is marked as `xfail` for `id=999`
(a post that doesn't exist)

Hint: `@pytest.mark.parametrize('post_id', [1, 2, 3, 5, 10])` — then use `post_id` inside the test.

Base URL: <https://jsonplaceholder.typicode.com> | No authentication required | Level 2 — builds on Cases 1-5